

Téléchargement de fichiers

Le système permet à chaque utilisateur de télécharger les fichiers stockés dans l'application, soit depuis la liste générale (réservée aux administrateurs), soit depuis la page Mes fichiers, accessible à tous les utilisateurs.

Pour cela, une route dédiée `app_telechargement_fichier` a été mise en place.

Cette route reçoit en paramètre l'identifiant du fichier à télécharger, ce qui permet ensuite au contrôleur d'effectuer plusieurs opérations :

- Récupération automatique du fichier grâce au `ParamConverter` de Symfony, qui injecte directement l'objet `Fichier` correspondant à l'ID.
- Vérification de l'existence du fichier en base : si aucun fichier ne correspond, l'utilisateur est redirigé.
- Construction du chemin complet du fichier sur le serveur grâce au paramètre `file_directory`.
- Téléchargement sécurisé via la méthode `$this->file()`, qui prépare et renvoie automatiquement une réponse HTTP adaptée, avec le bon nom original du fichier.

Au niveau de l'interface, chaque fichier est associé à un bouton ou un lien "Télécharger" utilisant cette route. L'utilisateur peut donc récupérer son fichier rapidement.

```
#[Route('/private-telechargement-fichier/{id}', name: 'app_telechargement_fichier', requirements:
    ["id" => "\d+"])
public function telechargementFichier(Fichier $fichier)
{
    if ($fichier == null) {
        $this->redirectToRoute('app_liste_fichiers_par_utilisateur');
    } else {
        return $this->file($this->getParameter('file_directory') . '/' . $fichier->getNomServeur(),
            $fichier->getNomOriginal());
    }
}
```

J'ai créé l'ajout des fichiers, qui est une fonctionnalité accessible à tous les utilisateurs

Elle permet d'ajouter un fichier qui vient directement de la machine de l'utilisateur

L'ajout de fichiers est une fonctionnalité accessible à tous les utilisateurs

Elle permet d'associer un fichier à un utilisateur et d'enregistrer plusieurs informations essentielles : le nom original du fichier, son nom sur le serveur, son extension (jpg, png, etc.), sa taille et enfin sa date d'envoi.

L'interface a été créée en Twig + Bootstrap, avec un formulaire Symfony généré par un `FormType` (`FichierType`).

Côté serveur, Symfony récupère le fichier envoyé et applique plusieurs étapes :

- Vérification du format via le composant `Symfony Validator`

- Génération d'un nom unique grâce au Slugger
- Déplacement sécurisé du fichier dans le répertoire configuré
- Sauvegarde automatique des métadonnées dans la base via Doctrine

Ce fonctionnement garantit un transfert de fichiers fiable, sécurisé et traçable.

Dans FichierType, plusieurs champs sont définis pour construire le formulaire d'ajout de fichier.

Le champ fichier utilise FileType avec une liste de contraintes de validation, notamment :

- maxSize pour limiter le poids du fichier
- mimeTypes pour restreindre les formats autorisés (PDF, JPEG, PNG)
- Un message personnalisé en cas d'erreur

Ce champ n'est pas mappé à l'entité (mapped => false), ce qui signifie que Symfony ne l'enregistre pas automatiquement et laisse le contrôleur gérer son traitement

Le champ user est un EntityType qui permet de sélectionner un utilisateur dans la base de données. Il affiche le nom complet grâce à une fonction choice_label et applique un style Bootstrap pour améliorer la présentation visuelle.

Le bouton ajouter est stylisé avec des classes Bootstrap pour également améliorer la présentation visuelle.

```
class FichierType extends AbstractType
{
    public function buildForm(FormBuilderInterface $builder, array $options): void
    {
        $builder
            ->add('fichier', FileType::class, array('label' => 'Fichier', 'mapped' => false, 'attr' => ['class' =>
                'form-control'], 'label_attr' => ['class' => 'fw-bold'], 'constraints' => [
                    new File([
                        'maxSize' => '200M',
                        'mimeTypes' => [
                            'application/pdf',
                            'application/x-pdf',
                            'image/jpeg',
                            'image/png',
                        ],
                    ]),
                    'mimeTypeMessage' => 'Le site accepte uniquement les fichiers PDF, PNG et JPG',
                ],
            )),
            ->add('user', EntityType::class, [
                'class' => User::class,
                'choice_label' => function ($user) {
                    return $user->getLastName() . ' ' . $user->getFirstname();
                },
                'label' => 'Utilisateur',
                'label_attr' => ['class' => 'fw-bold'],
                'attr' => ['class' => 'form-control'],
                'row_attr' => ['class' => 'mb-3'],
            ]),
            ->add('ajouter', SubmitType::class, [
                'attr' => ['class' => 'btn bg-primary text-white m-4'],
                'row_attr' => ['class' => 'text-center'],
            ])
        ;
    }
}
```

Liste générale des fichiers réservée aux administrateurs

La liste générale des fichiers est une fonctionnalité réservée aux administrateurs.

Elle permet d'afficher l'ensemble des fichiers stockés dans l'application, avec toutes leurs informations importantes : nom original, extension, taille, date d'envoi ainsi que l'utilisateur qui les a ajoutés.

Cette fonctionnalité repose sur une méthode simple dans le contrôleur :

- Une route dédiée (app_liste_fichiers) permet d'accéder à la page.
- Le contrôleur utilise le FichierRepository pour récupérer tous les fichiers via findAll().
- L'ensemble des fichiers est ensuite transmis au template Twig liste_fichiers.html.twig.

Ce fonctionnement garantit une vue complète de tous les fichiers présents dans la base de données.

Côté interface, un tableau responsive en Twig + Bootstrap affiche les données importantes pour chaque fichier, ainsi qu'un bouton permettant de le télécharger directement grâce à la route prévue à cet effet vue juste au-dessus.

```
#[Route('/liste-fichiers', name: 'app_liste_fichiers')]
public function listeFichiers(FichierRepository $fichierRepository): Response
{
    $fichiers = $fichierRepository->findAll();

    return $this->render('fichier/liste_fichiers.html.twig', [
        'fichiers' => $fichiers,
    ]);
}
```

Affichage des fichiers personnels (Mes fichiers)

La fonctionnalité *Mes fichiers* permet à chaque utilisateur connecté d'accéder à tous les fichiers qu'il a envoyés.

Cette page est accessible quel que soit le rôle (utilisateur ou administrateur) et offre un espace où l'utilisateur peut visualiser et télécharger ses propres documents.

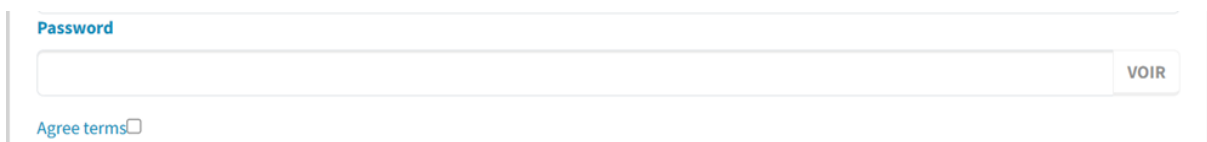
Sur le plan technique, la route /mes-fichiers appelle une méthode du contrôleur qui utilise le *FichierRepository* afin de récupérer uniquement les fichiers appartenant à l'utilisateur actuellement connecté. Pour cela, Symfony utilise la fonction `getUser()` qui renvoie l'utilisateur authentifié, puis interroge la base via une recherche conditionnée sur ce dernier.

Une seconde requête, réalisée à l'aide d'un Query Builder personnalisé (`getTotalStorageUsed()`), permet de calculer automatiquement la taille totale de stockage utilisée par l'utilisateur. Cette information est renvoyée à Twig afin d'être affichée sur l'interface.

Enfin, le rendu est effectué via un template dédié (`mes-fichiers.html.twig`) qui affiche les fichiers sous forme de tableau : nom, extension, taille, date d'envoi, ainsi qu'un bouton Télécharger pour chaque entrée.

```
#[Route('/mes-fichiers', name: 'app_mes_fichiers')]
public function mesFichiers(FichierRepository $fichierRepository): Response
{
    $user = $this->getUser();
    $fichiers = $fichierRepository->findBy(
        ['user' => $user],
        ['dateEnvoi' => 'DESC']
    );
    $totalUsed = $fichierRepository->getTotalStorageUsed($user);
    return $this->render('fichier/mes-fichiers.html.twig', [
        'fichiers' => $fichiers,
        'totalUsed' => $totalUsed,
    ]);
}
```

Fonction js

A screenshot of a web form. At the top, the word "Password" is written in blue. Below it is a white input field with a light blue border. To the right of the input field is a small button with the text "VOIR" in blue. Below the input field, there is a link "Agree terms" followed by a small square checkbox.

Pour améliorer l'expérience utilisateur lors de la saisie du mot de passe, j'ai mis en place une fonctionnalité JavaScript permettant d'afficher ou de masquer le mot de passe dans les formulaires d'inscription et de changement de mot de passe.

Un bouton placé à droite du champ de saisie permet à l'utilisateur d'alterner entre deux états :

- Mode caché : le mot de passe apparaît sous forme de point.
- Mode visible : le texte s'affiche en clair pour éviter les erreurs de saisie.

Le script écoute simplement l'événement *click* sur le bouton "Voir / Cacher" et modifie le type du champ. Le texte du bouton se met également à jour pour indiquer l'action inverse ("Voir" à "Cacher").

Cette fonctionnalité est entièrement réalisée côté client et n'a aucun impact sur la sécurité serveur : elle améliore uniquement l'ergonomie du formulaire, facilitant la saisie d'un mot de passe complexe.

```
3 document.addEventListener("DOMContentLoaded", () => {
4     const passwordInput = document.getElementById("registration_form_plainPassword");
5     const toggleBtn = document.getElementById("toggle-password");
6
7     if (!passwordInput || !toggleBtn) {
8         console.error("Password toggle: élément introuvable.");
9         return;
10    }
11
12    toggleBtn.addEventListener("click", () => {
13        const isHidden = passwordInput.type === "password";
14
15        passwordInput.type = isHidden ? "text" : "password";
16        toggleBtn.textContent = isHidden ? "Cacher" : "Voir";
17    });
18 });
19
```

Firstname

Lastname

Adresse

Ville

Code postal

Email

Password

CACHER

Fort

Agree terms☒

S'INSCRIRE

Firstname

Lastname

Adresse

Ville

Code postal

Email

Password

VOIR

Agree terms☐

S'INSCRIRE

Activation du bouton d'inscription uniquement lorsque toutes les données sont valides

Pour renforcer la fiabilité et la sécurité du formulaire d'inscription, j'ai mis en place un script JavaScript permettant de désactiver le bouton d'inscription tant que toutes les informations saisies ne sont pas correctes.

Ce système évite l'envoi d'un formulaire incomplet ou contenant des données invalides, ce qui améliore l'expérience utilisateur et la qualité des données stockées.

Dès le chargement de la page, le script récupère chaque champ du formulaire (le nom, le prénom, l'adresse, la ville, le code postal, l'email, le mot de passe et la case "J'accepte les conditions").

Plusieurs vérifications sont ensuite effectuées en temps réel :

- Contrôle de la complétude : aucun champ ne doit être vide.
- Validation de l'email : utilisation d'une expression régulière pour vérifier que l'adresse est au bon format.
- Validation du mot de passe : au moins 12 caractères, incluant majuscules, minuscules, chiffres et caractères spéciaux.
- Vérification du consentement : la case des conditions générales doit être cochée.

Si une de ces conditions n'est pas remplie, le bouton "S'inscrire" reste désactivé et visuellement grisé. Lorsque toutes les règles sont respectées, le bouton s'active automatiquement, permettant l'envoi du formulaire.

```

assets > js > JS form-validation.js > document.addEventListener("DOMContentLoaded") callback
1  document.addEventListener("DOMContentLoaded", () => {
2      const firstname = document.querySelector("#registration_form_firstname");
3      const lastname = document.querySelector("#registration_form_lastname");
4      const adresse = document.querySelector("#registration_form_adresse");
5      const ville = document.querySelector("#registration_form_ville");
6      const codePostal = document.querySelector("#registration_form_code_postal");
7      const email = document.querySelector("#registration_form_email");
8      const password = document.querySelector("#registration_form_plainPassword");
9      const agreeTerms = document.querySelector("#registration_form_agreeTerms");
10     const submitBtn = document.querySelector("button[type='submit']");
11     submitBtn.disabled = true;
12     submitBtn.classList.add("disabled");
13
14     function isValidEmail(value) {
15         return /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(value);
16     }
17
18     function isValidPassword(value) {
19         return (
20             value.length >= 12 &&
21             /[A-Z]/.test(value) &&
22             /[a-z]/.test(value) &&
23             /[0-9]/.test(value) &&
24             /[\\W]/.test(value)
25         );
26     }
27
28     function checkForm() {
29         const isComplete =
30             firstname.value.trim() !== "" &&
31             lastname.value.trim() !== "" &&
32             adresse.value.trim() !== "" &&
33             ville.value.trim() !== "" &&
34             codePostal.value.trim() !== "" &&
35             email.value.trim() !== "" &&
36             password.value.trim() !== "" &&
37             agreeTerms.checked;
38
39         const isValid =
40             isComplete &&
41             isValidEmail(email.value) &&
42             isValidPassword(password.value);
43
44         submitBtn.disabled = !isValid;
45
46         if (isValid) {
47             submitBtn.classList.remove("disabled");
48         } else {
49             submitBtn.classList.add("disabled");
50         }

```


The image displays three examples of a password strength indicator. Each example consists of a text input field, a label 'Password' above it, and a 'VOIR' button to its right. Below the input field is a horizontal bar representing the strength score, with the word 'Faible', 'Moyen', or 'Fort' written below the bar.

- Example 1:** The input field contains '...'. The bar is red and short. The label below the bar is 'Faible'.
- Example 2:** The input field contains '.....'. The bar is orange and medium-length. The label below the bar is 'Moyen'.
- Example 3:** The input field contains '.....'. The bar is green and full-length. The label below the bar is 'Fort'.

Barre de force du mot de passe

Pour améliorer la sécurité et guider l'utilisateur lors de la création de son mot de passe, j'ai intégré une barre dynamique indiquant en temps réel la solidité du mot de passe saisi.

Cette fonctionnalité repose entièrement sur JavaScript et s'active à chaque frappe dans le champ du mot de passe. Le script analyse plusieurs critères : la longueur, la présence de chiffres, de lettres majuscules et de caractères spéciaux. En fonction du nombre de critères remplis, un score est calculé.

Ce score met automatiquement à jour une barre colorée (rouge, orange ou verte) ainsi qu'un message indiquant le niveau de sécurité : faible, moyen ou fort.

Ce retour visuel instantané aide l'utilisateur à choisir un mot de passe plus robuste et renforce la sécurité globale du système, notamment lors de l'inscription et du changement de mot de passe.

```

document.addEventListener("DOMContentLoaded", () => {

    const passwordInput = document.getElementById("registration_form_plainPassword");
    const strengthBar = document.getElementById("password-strength-bar");
    const strengthText = document.getElementById("password-strength-text");

    if (!passwordInput || !strengthBar || !strengthText) {
        console.warn("Password strength: élément introuvable.");
        return;
    }

    passwordInput.addEventListener("input", () => {
        const value = passwordInput.value;
        let score = 0;

        if (value.length >= 6) score++;
        if (/[0-9]/.test(value)) score++;
        if (/[A-Z]/.test(value)) score++;
        if (/^[A-Za-z0-9]/.test(value)) score++;
        strengthBar.style.height = "6px";

        switch (score) {
            case 0:
            case 1:
                strengthBar.style.width = "33%";
                strengthBar.style.backgroundColor = "red";
                strengthText.textContent = "Faible";
                break;

            case 2:
            case 3:
                strengthBar.style.width = "66%";
                strengthBar.style.backgroundColor = "orange";
                strengthText.textContent = "Moyen";
                break;

            case 4:
                strengthBar.style.width = "100%";
                strengthBar.style.backgroundColor = "green";
                strengthText.textContent = "Fort";
                break;
        }
    });
});

```

Barre de force du mot de passe

J'ai ajouté une barre qui indique en temps réel la solidité du mot de passe lors de sa saisie.

Cette barre change de couleur et affiche un niveau de sécurité pour aider l'utilisateur à créer un mot de passe plus fiable.

Ce retour visuel rend la création de mot de passe plus intuitive et encourage l'utilisation de mots de passe plus robustes, aussi bien à l'inscription que lors d'un changement de mot de passe.

Récupérer les fichiers selon un utilisateur précis Pour cette requête, j'ai créé une méthode personnalisée dans le FichierRepository afin de récupérer uniquement les fichiers appartenant à un utilisateur précis. L'objectif est de ne plus dépendre du simple findBy(), mais d'utiliser le Query Builder de Doctrine, qui offre un contrôle plus fin sur la construction de la requête. La méthode findFilesByUser() commence par créer un alias f représentant l'entité Fichier. Ensuite, la requête ajoute une clause WHERE afin de sélectionner uniquement les fichiers dont la colonne user correspond à l'utilisateur passé en paramètre. Un paramètre sécurisé est utilisé (:user) afin d'éviter toute injection SQL. La méthode termine par un tri des résultats grâce à orderBy(), qui classe les fichiers du plus récent au plus ancien selon le champ dateEnvoi. Enfin, la requête est exécutée à l'aide de getQuery()->getResult() qui renvoie un tableau d'objets Fichier. Cette requête est utilisée dans la fonctionnalité Mes fichiers, permettant à chaque utilisateur de visualiser uniquement ses propres fichiers, avec un tri chronologique.

```
public function findFilesByUser($user): array
{
    return $this->createQueryBuilder('f')
        ->andWhere('f.user = :user')
        ->setParameter('user', $user)
        ->orderBy('f.dateEnvoi', 'DESC')
        ->getQuery()
        ->getResult();
}
```

```
public function getTotalStorageUsed($user)
{
    return $this->createQueryBuilder('f')
        ->select('SUM(f.taille)')
        ->andWhere('f.user = :user')
        ->setParameter('user', $user)
        ->getQuery()
        ->getSingleScalarResult();
}
```

 **Mes fichiers**

Stockage utilisé : 183.88 Ko (0.18 Mo)

Fichiers appartenant à mon compte				
Fichier	Extension	Taille	Date d'envoi	Télécharger
chapitre02-symfony64.pdf	pdf	91.94 Ko	11/12/2025 18:38	TÉLÉCHARGER
chapitre02-symfony64.pdf	pdf	91.94 Ko	11/12/2025 18:34	TÉLÉCHARGER
deqsiom	jpg	0.00 Ko	01/01/2020 00:00	TÉLÉCHARGER

Calcul de l'espace total utilisé par un utilisateur Cette requête permet de déterminer combien d'espace de stockage un utilisateur a consommé sur la plateforme. Pour cela, j'ai ajouté dans le FichierRepository une méthode utilisant le Query Builder de Symfony. Elle sélectionne la somme de la taille de tous les fichiers appartenant à un utilisateur donné, en utilisant la fonction SQL d'agrégation SUM(). La requête applique également une condition (WHERE f.user = :user) afin de calculer uniquement les tailles liées à l'utilisateur actuellement connecté. La méthode utilise getSingleScalarResult(), ce qui permet de récupérer une valeur simple (un nombre représentant le total en octets). Cette donnée est ensuite envoyée à la vue Twig, où elle est formatée et affichée sous différentes unités (Ko, Mo).

```
public function findTopUsersByFileCount(int $limit = 5): array
{
    return $this->createQueryBuilder('f')
        ->select('u.id, u.firstname, u.lastname, COUNT(f.id) AS fileCount')
        ->leftJoin('f.user', 'u')
        ->groupBy('u.id')
        ->orderBy('fileCount', 'DESC')
        ->setMaxResults($limit)
        ->getQuery()
        ->getResult();
}
```

```
#[Route('/top-users', name: 'app_top_users')]
public function topUsers(FichierRepository $repo): Response
{
    $topUsers = $repo->findTopUsersByFileCount(5);

    return $this->render('fichier/top-users.html.twig', [
        'topUsers' => $topUsers,
    ]);
}
```

Navbar

HomeAdministrationActivité GlobaleLa liste des fichiersMots de passe expirésTop usersMon ActivitéAjouter des fichiersMes fichiersAbout

TEST NOTIFICATION

49

MODE SOMBRE

sqkljfd@qdmstfk.dfsj

 **Top utilisateurs — Envois de fichiers**

Classement des utilisateurs

#	Utilisateur	Nombre de fichiers
1	sdqkjl sqjd	3
2	parmentier leo	1
3	salingue ethan	1
4	ethan clem	1
5	gobain ethan	1

Affichage des utilisateurs qui envoient le plus de fichiers

Pour exploiter la requête Query Builder permettant d'identifier les utilisateurs ayant envoyé le plus de fichiers, j'ai ajouté une nouvelle route Symfony (/top-users) associée à un contrôleur dédié.

Ce contrôleur appelle la méthode `findTopUsersByFileCount()` du `FichierRepository`, qui retourne une liste triée des utilisateurs accompagnée du nombre total de fichiers qu'ils ont uploadés. Cette requête utilise une jointure avec l'entité `User`, un `GROUP BY` et une fonction d'agrégation `COUNT()` pour générer un classement fiable.

Une fois les résultats récupérés, ils sont transmis à une vue Twig (`top-users.html.twig`) où ils peuvent être affichés sous forme de tableau ou de liste, permettant ainsi de consulter facilement les utilisateurs les plus actifs sur la plateforme.

 **Utilisateurs n'ayant pas changé leur mot de passe depuis 2 jours**

Liste des utilisateurs concernés

Email	Nom	Dernier changement
qsfdmjqs@qsdgimif.csoifae	bgd thomas	19/11/2025 à 21:31
mkleqfjds@dqsfimj.dqsfoij	bgd leo	19/11/2025 à 18:54
qfidsmj@sdqmlkf.qdsmfli	ethan clem	19/11/2025 à 20:42
sdqf@dsqf.dfsq	sdq clem	19/11/2025 à 21:43
sqkljfd@qdmstfk.dfsj	sdqkjl sqjd	19/11/2025 à 21:46
ethan.gobain@gmail.com	gobain ethan	19/11/2025 à 21:50

Trouver les utilisateurs n'ayant pas changé leur mot de passe depuis X jours

Cette requête permet d'identifier automatiquement les utilisateurs dont le mot de passe n'a pas été modifié depuis un certain nombre de jours.

Elle repose sur la table `password_history`, qui enregistre chaque changement de mot de passe d'un utilisateur avec la date exacte de la modification.

Pour effectuer cette recherche, j'ai ajouté une méthode dédiée dans le *UserRepository*.

Le Query Builder réalise plusieurs opérations importantes :

- Jointure entre `user` et `password_history` pour accéder aux dates de changement de mot de passe.
- Groupement par utilisateur afin de récupérer correctement les données lorsqu'un utilisateur possède plusieurs anciennes entrées.
- Utilisation d'une fonction d'agrégation (MAX) pour récupérer la date la plus récente du changement de mot de passe.
- Filtre avec HAVING pour n'afficher que les utilisateurs dont la dernière modification est antérieure à la limite définie par exemple 2 jours.
- Paramétrage dynamique permettant de modifier la valeur X facilement.

Cette requête est utilisée dans une nouvelle route dédiée (`/old-passwords`) permettant aux administrateurs d'afficher la liste des comptes concernés via une page Twig.